

TP N1 — Du fichier aux tokens

Module NLP, séance 1 — projet **ILIACode**

Travail Individuel

Pr. Asmaa RASSIL

ENSA Fès, Filière ILIA (3^e année)

Le code couleur du document

Chaque nature d'objet porte sa couleur, la même d'un bout à l'autre du TP.

- **texte.py** est un fichier ou un dossier.
- **lire** est une des cinq fonctions à compléter.
- **ftfy** est une bibliothèque ou un objet importé.
- **slide 7** renvoie au cours de la séance.
- **notion importante**, **erreur ou test en échec**, **test au vert**.
- commande tapée ou affichage de l'écran, sans nature particulière.

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Lire
- 3 Partie 2 — Réparer
- 4 Partie 3 — Normaliser
- 5 Partie 4 — Tokeniser
- 6 Partie 5 — Mesurer
- 7 Partie 6 — Publier

Partie 0 — La mise en place

Lien avec le cours

Cette partie suit les **slides 2** (Le premier maillon du copilote ILIACode), **3** (La chaîne du fichier aux tokens) et **4** (Le déroulement de la séance). Elle prépare les deux fichiers annoncés au **slide 4**.

Étapes 0.1 et 0.2 — Le dossier de la séance

Étape 0.1. Placer le dossier de la séance dans votre dépôt.

Faites un clic droit sur **seance1_etudiants.zip**, choisissez *Extraire tout*, et placez le dossier **seance1** obtenu **à l'intérieur** de votre dossier **iliacode**, celui créé au TP 0.

Étape 0.2. Ouvrir le dépôt dans VS Code.

Lancez VS Code, menu *Fichier, Ouvrir le dossier*, et choisissez **iliacode**. Dans la colonne de gauche, déployez **seance1**. Vous devez y voir **texte.py**, **test_texte.py**, **mesure_corpus.py** et un dossier **donnees**.

Étapes 0.3 et 0.4 — Le terminal et les bibliothèques

Étape 0.3. Ouvrir le terminal et entrer dans le dossier.

Menu *Terminal*, *Nouveau terminal*. Activez votre environnement virtuel exactement comme au TP 0, son nom doit apparaître entre parenthèses au début de la ligne.

```
cd iliacode  
.venv\Scripts\activate.bat
```

Puis tapez la commande suivante.

```
cd seance1
```

Étape 0.4. Installer les bibliothèques de la séance.

```
pip install ftfy pytest charset-normalizer
```

ftfy répare les textes abîmés, **pytest** lance les tests et **charset-normalizer** devine la règle d'écriture d'un fichier.

Étape 0.5 — Les 13 tests une première fois

Étape 0.5. Lancer les 13 tests une première fois.

```
pytest -v
```

Vérification avant de continuer

Treize lignes se terminent par **FAILED**, et la dernière ligne de l'écran est la suivante (le temps affiché varie).

```
===== 13 failed in 0.10s =====
```

C'est **normal**. Les cinq fonctions sont encore vides, donc tous les tests échouent. La séance consiste à faire passer ces 13 lignes au vert.

Étape 0.6 — Le fichier à compléter

Étape 0.6. Regarder le fichier à compléter.

Cliquez sur **texte.py**. Il contient cinq fonctions, **lire**, **reparer**, **normaliser**, **tokeniser** et **mesurer**, une par étape de la chaîne du **slide 3**. Chacune se termine par une ligne qui commence par `raise NotImplementedError`. Cette ligne veut dire « pas encore écrit ». Compléter une fonction, c'est **remplacer cette ligne** par le vrai code.

Règle du parcours

Après chaque modification du fichier, enregistrez avec **Ctrl+S** avant de lancer les tests.

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Lire**
- 3 Partie 2 — Réparer
- 4 Partie 3 — Normaliser
- 5 Partie 4 — Tokeniser
- 6 Partie 5 — Mesurer
- 7 Partie 6 — Publier

Lien avec le cours

Cette partie applique les **slides 6** (Les octets et les caractères), **7** (La lecture d'un fichier) et **8** (Les fichiers écrits avant UTF-8). Les étapes 1.1 et 1.2 rejouent le cadre gris du **slide 7**, l'étape 1.3 est son encadré Application, l'étape 1.5 rejoue le tableau du **slide 8**.

Étape 1.1 — L'interpréteur Python

Étape 1.1. Ouvrir l'interpréteur Python (slide 7).

Chaque ligne tapée est exécutée tout de suite et sa réponse s'affiche.

```
python
```

Le début de ligne devient `>>>`. Tant que vous voyez `>>>`, vous êtes dans l'interpréteur. Dans les cadres gris qui suivent, ne tapez pas les `>>>`, tapez seulement ce qui vient après.

Étape 1.2 — D'une forme à l'autre

Étape 1.2. Passer d'une forme à l'autre (slides 6 et 7).

Tapez ces quatre lignes une par une. Sous chacune, l'écran doit afficher la ligne sans >>>.

```
>>> octets = "créé".encode("utf-8")
>>> octets
b'cr\xc3\xa9\xc3\xa9'
>>> len(octets)
6
>>> texte = octets.decode("utf-8")
>>> texte
'créé'
>>> len(texte)
4
```

Étape 1.2 — D'une forme à l'autre (suite)

Vous venez de faire les deux flèches du schéma du **slide 6**. **encode** va des caractères vers les octets, **decode** revient des octets vers les caractères. Le même mot mesure **6 octets** et **4 caractères**, parce que chaque « é » occupe 2 octets, \xc3\xa9.

Quittez maintenant l'interpréteur.

```
>>> exit()
```

Étape 1.3 — La fonction `lire`

Étape 1.3. Compléter la fonction `lire` (slide 7, encadré Application).

Dans `texte.py`, trouvez la fonction `lire`. Effacez sa ligne raise `NotImplementedError(...)` et écrivez à la place les trois lignes ci-dessous, en respectant les espaces du début.

```
with open(chemin, "rb") as f:
    octets = f.read()
return octets.decode(regle)
```

Étape 1.3 — La fonction `lire` (suite)

- `open(chemin, "rb")` ouvre le fichier en mode binaire, *read bytes*. Le disque est rendu tel quel, en octets.
- `f.read()` prend tous les octets du fichier.
- `octets.decode(rgle)` les décode avec la règle reçue, UTF-8 si rien n'est précisé, et `return` renvoie le texte obtenu.

Enregistrez avec **Ctrl+S**.

Étape 1.4 — Les deux tests de la partie

Étape 1.4. Lancer les deux tests de la partie.

L'option `-k lire` ne lance que les tests dont le nom contient `lire`.

```
pytest -v -k lire
```

Vérification avant de continuer

```
test_texte.py::test_lire_utf8 PASSED
test_texte.py::test_lire_ancien_encodage PASSED
===== 2 passed, 11 deselected in 0.12s =====
```

Étape 1.5 — Un fichier écrit avant UTF-8

Étape 1.5. Lire un fichier écrit avant UTF-8 (slide 8).

Le fichier **donnees/ticket_1256.txt** a été écrit avec la règle **Windows-1256**, nommée **cp1256** en Python. Relancez python, puis essayez les trois lignes du tableau du **slide 8**.

```
>>> octets = open("donnees/ticket_1256.txt", "rb").read()
>>> octets
b'\xe3\xe1\xdd'
>>> octets.decode("latin-1")
'ăáÝ'
>>> octets.decode("utf-8")
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe3 in position 0: ...
>>> octets.decode("cp1256")
```

Étape 1.5 — Un fichier écrit avant UTF-8 (suite)

La dernière ligne affiche le mot arabe ملف, dossier. Les trois essais lisent **les mêmes octets**. Avec la bonne règle le mot est juste, avec Latin-1 il est **faux sans aucune erreur**, avec UTF-8 Python refuse.

En cas de problème

Le mot arabe peut s'afficher avec des lettres détachées ou dans l'ordre inverse. C'est une limite d'affichage du terminal, le contenu est juste, et le test **test_lire_ancien_encodage** de l'étape 1.4 le prouve.

Dans une autre application

Type d'application. Un assistant interne pour une banque, une assurance ou une administration, qui doit lire des archives écrites avant UTF-8.

Comment. Le fichier **donnees/archive_1256.txt** est une de ces archives. Restez dans l'interpréteur et tapez les lignes suivantes.

```
>>> from charset_normalizer import from_bytes
>>> octets = open("donnees/archive_1256.txt", "rb").read()
>>> from_bytes(octets).best().encoding
'cp1256'
>>> from texte import lire
>>> texte = lire("donnees/archive_1256.txt", regle="cp1256")
>>> open("donnees/archive_utf8.txt", "w", encoding="utf-8").write(texte)
345
>>> exit()
```

Partie 1 — Lire (suite)

Dans une autre application

La bibliothèque `charset-normalizer` a proposé la règle cp1256 d'après les octets, votre fonction `lire` a décodé l'archive, et la dernière ligne l'a réécrite en UTF-8. C'est le geste d'entrée de toute application sur archives. **On convertit une fois, puis tout le reste de l'application ne voit que de l'UTF-8.**

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Lire
- 3 Partie 2 — Réparer**
- 4 Partie 3 — Normaliser
- 5 Partie 4 — Tokeniser
- 6 Partie 5 — Mesurer
- 7 Partie 6 — Publier

Lien avec le cours

Cette partie applique les **slides 9** (Le mojibake) et **10** (La réparation du mojibake). L'étape 2.2 rejoue les actes 2 et 3 du tableau du **slide 9**, l'étape 2.3 est le cadre gris du **slide 10**, l'étape 2.5 son encadré Application.

Étape 2.1 — Le problème sur la première ligne du fil

Étape 2.1. Voir le problème sur la première ligne du fil (slide 9).

Relancez python, puis lisez la ligne de code avec votre fonction.

```
>>> from texte import lire
>>> ligne = lire("donnees/ligne_code.txt")
>>> ligne
'def is_valid_email(adresse): # crÃ©Ã© par K. Alaoui\n'
```

Le fichier est lu avec la bonne règle, et pourtant il contient « **crÃ©Ã©** ». C'est le Problème du **slide 9**. Le `\n` final est le retour à la ligne.

Étapes 2.2 et 2.3 — L'accident et son inverse

Étape 2.2. Reproduire l'accident (slide 9, actes 1 et 2).

```
>>> abime = "créé".encode("utf-8").decode("latin-1")
>>> abime
'crÃ©Ã©'
```

Le mot a été encodé avec une règle et décodé avec une autre. C'est la définition du mojibake.

Étape 2.3. Inverser l'accident (slide 10, cadre gris).

Les deux mêmes règles sont appliquées dans l'ordre contraire.

```
>>> abime.encode("latin-1").decode("utf-8")
'créé'
```

Étape 2.4 — La réparation par `ftfy`

Étape 2.4. Laisser `ftfy` le faire sur une ligne entière (**slide 10**).

```
>>> import ftfy
>>> ftfy.fix_text(ligne, normalization=None)
'def is_valid_email(adresse): # créé par K. Alaoui\n'
>>> exit()
```

Étape 2.5 — La fonction `reparer`

Étape 2.5. Compléter la fonction `reparer` (slide 10, encadré Application).

Dans `texte.py`, remplacez la ligne raise de la fonction `reparer` par une seule ligne. Elle commence par `return` et renvoie exactement ce que vous venez de taper à l'étape 2.4, avec `texte` à la place de `ligne`.

```
return ftfy.fix_text(_____, normalization=None)
```

Remplacez les tirets par le bon nom, puis enregistrez.

Étape 2.6 — Les deux tests de la partie

Étape 2.6. Lancer les deux tests de la partie.

```
pytest -v -k reparer
```

Vérification avant de continuer

```
test_texte.py::test_reparer_mojibake PASSED
test_texte.py::test_reparer_laisse_le_texte_sain PASSED
===== 2 passed, 11 deselected in 0.12s =====
```

Le second test donne un texte déjà correct et vérifie qu'il ressort intact. **Une réparation ne doit jamais abîmer un texte sain.**

Dans une autre application

Type d'application. Un assistant de relation client, de ressources humaines ou d'hôpital, qui cherche des personnes par leur nom dans une base passée par plusieurs anciens logiciels.

Comment. Relancez python. La deuxième ligne fabrique une fiche abîmée comme celles de ces bases, la quatrième la répare.

```
>>> from texte import reparer
>>> fiche = "Client Benoît Lefèvre".encode("utf-8").decode("latin-1")
>>> fiche
'Client BenoÃ®t LefÃvre'
>>> reparer(fiche)
'Client Benoît Lefèvre'
>>> exit()
```

Partie 2 — Réparer (suite)

Dans une autre application

Sans cette passe, **chercher « Benoît » ne trouve jamais la fiche**. On applique donc **reparer** à chaque champ texte à l'entrée, **avant de stocker et d'indexer**.

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Lire
- 3 Partie 2 — Réparer
- 4 Partie 3 — Normaliser**
- 5 Partie 4 — Tokeniser
- 6 Partie 5 — Mesurer
- 7 Partie 6 — Publier

Lien avec le cours

Cette partie applique les **slides 12** (Les formes normales Unicode), **13** (L'écriture arabe et la darija) et **14** (La normalisation en pratique). L'étape 3.1 rejoue le Problème du **slide 12**, l'étape 3.2 le tableau NFC et NFKC du **slide 12**, l'étape 3.3 le cadre gris du **slide 13**, l'étape 3.4 est l'encadré Application du **slide 14**.

Étape 3.1 — Deux textes identiques à l'écran

Étape 3.1. Voir deux textes identiques à l'écran et différents pour Python (slide 12).

Relancez python. Dans une chaîne, `\u00e9` désigne le caractère de numéro U+00E9, le « é » en un seul caractère, et `\u0301` désigne l'accent seul.

```
>>> a = "cr\u00e9\u00e9"
>>> b = "cre\u0301e\u0301"
>>> a, b
('créé', 'créé')
>>> a == b
False
>>> len(a), len(b)
(4, 6)
```

Étape 3.2 — La normalisation

Étape 3.2. Normaliser (slides 12 et 14).

```
>>> import unicodedata
>>> unicodedata.normalize("NFKC", a) == unicodedata.normalize("NFKC", b)
True
>>> unicodedata.normalize("NFKC", "m\u00b2")
'm2'
>>> unicodedata.normalize("NFC", "m\u00b2")
'm²'
```

\u00b2 est l'exposant «²». **NFKC** le change en 2, **NFC** le garde. C'est la seule différence entre les deux formes, celle du tableau du **slide 12**.

Étape 3.3 — Les gestes arabes

Étape 3.3. Les gestes arabes (slide 13, cadre gris).

Le mot كَتَبَ, écrit avec ses trois voyelles courtes, compte 6 caractères. Il est écrit ici par ses numéros pour éviter de changer de clavier.

```
>>> import re
>>> mot = "\u0643\u064e\u062a\u064e\u0628\u064e"
>>> len(mot)
6
>>> len(re.sub("[\u064B-\u065F]", "", mot))
3
>>> exit()
```

La ligne `re.sub` a supprimé les trois voyelles, il reste les trois lettres de كَتَبَ.

Étape 3.4 — La fonction `normaliser`

Étape 3.4. Compléter la fonction `normaliser` (slide 14, encadré Application).

Remplacez la ligne raise de `normaliser` par les cinq lignes suivantes. La première est à compléter, c'est la ligne de l'étape 3.2 avec les deux arguments de la fonction, `forme` et `texte`. Les trois suivantes sont les trois gestes arabes du slide 13.

```
texte = unicodedata.normalize(_____, _____)
texte = re.sub("[\u064B-\u065F]", "", texte)
texte = texte.replace("\u0640", "")
texte = texte.translate(CHIFFRES_ARABES)
return texte
```

`CHIFFRES_ARABES` est déjà défini en haut du fichier, c'est la table qui associe chaque chiffre arabe à son chiffre latin. Enregistrez.

Étape 3.5 — Les quatre tests de la partie

Étape 3.5. Lancer les quatre tests de la partie.

```
pytest -v -k normaliser
```

Vérification avant de continuer

```
test_texte.py::test_normaliser_double_ecriture PASSED
test_texte.py::test_normaliser_compatibilite PASSED
test_texte.py::test_normaliser_arabe PASSED
test_texte.py::test_normaliser_forme_nfc_garde_l_exposant PASSED
===== 4 passed, 9 deselected in 0.12s =====
```

Lancez ensuite pytest sans option. La dernière ligne doit annoncer **5 failed**, **8 passed**.

Dans une autre application

Type d'application. Un assistant pour la santé ou les sciences, où un exposant porte du sens.

Comment. Relancez python et comparez les deux formes sur un dosage.

```
>>> from texte import normaliser
>>> normaliser("10\u00b2 mg")
'102 mg'
>>> normaliser("10\u00b2 mg", forme="NFC")
'102 mg'
>>> exit()
```

Partie 3 — Normaliser (suite)

Dans une autre application

Avec NFKC, «dix au carré milligrammes» devient «cent deux milligrammes», ce qui est une erreur grave. Dans ces applications on appelle donc **normaliser** avec **forme="NFC"**. Dans tous les cas, la **même** fonction est appliquée aux documents et aux demandes des utilisateurs.

— Pause de 15 minutes —

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Lire
- 3 Partie 2 — Réparer
- 4 Partie 3 — Normaliser
- 5 Partie 4 — Tokeniser**
- 6 Partie 5 — Mesurer
- 7 Partie 6 — Publier

Lien avec le cours

Cette partie applique les **slides 16** (Le token et la découpe sur les espaces), **17** (L'expression régulière), **18** (Les six règles du tokeniseur) et **19** (Le motif complet du tokeniseur). L'étape 4.1 rejoue la Tentative du **slide 16**, l'étape 4.2 le tableau des symboles du **slide 17**, l'étape 4.3 écrit d'un coup le motif du **slide 19**, ses cinq premières règles, et l'étape 4.4 ajoute la règle 6, qui ne s'écrit pas dans le motif.

Étape 4.1 — L'échec de la découpe sur les espaces

Étape 4.1. Voir l'échec de la découpe sur les espaces (slide 16).

Relancez python.

```
>>> "l'app t7bes, surface".split()  
["l'app", 't7bes,', 'surface']
```

La virgule est restée collée à t7bes. C'est la faille du **slide 16**.

Étape 4.2 — Trois expressions régulières

Étape 4.2. Essayer trois expressions régulières (slide 17).

Le `r` devant le guillemet dit à Python de ne pas interpréter les `\`.

```
>>> import re
>>> re.findall(r"\d+", "version 3 du 14 mars 2026")
['3', '14', '2026']
>>> re.findall(r"\w+", "l'app t7bes, surface")
['l', 'app', 't7bes', 'surface']
>>> re.findall(r"\w+|[\^\w\s]", "l'app t7bes, surface")
['l', "'", 'app', 't7bes', ',', ' ', 'surface']
>>> exit()
```

`\d+` reconnaît les suites de chiffres. `\w+` reconnaît les mots mais perd la ponctuation. Le dernier motif ajoute, après la barre `|`, « tout signe qui n'est ni une lettre ni un blanc », et la ponctuation revient, chaque signe à part. Ce dernier motif est le **cœur** du tokeniseur, les règles 4 et 5 du **slide 18**.

Étape 4.3 — Le motif des cinq premières règles

Étape 4.3. Écrire le motif des cinq premières règles (slide 19).

En haut de `texte.py`, la ligne `MOTIF` ne contient que le cœur. Remplacez-la par la ligne suivante, qui place les trois protections à gauche de ce cœur. Vous pouvez supprimer le commentaire qui la suit.

```
MOTIF = r' "[^"]*" | #.* | \d+ \. \d+ | != | <= | >= | \w+ | [^\w\s] '
```

Les alternatives se lisent de gauche à droite dans l'ordre des règles du slide 18, la chaîne et le commentaire de la règle 1, le nombre à virgule de la règle 2, les trois opérateurs composés de la règle 3, puis le cœur, le mot de la règle 4 et le signe isolé de la règle 5. Dans la fonction `tokeniser`, remplacez la ligne raise par cette ligne.

```
return re.findall(MOTIF, texte)
```

Étape 4.3 — Le motif des cinq premières règles (suite)

Enregistrez, puis lancez les tests de la partie.

```
pytest -v -k tokeniser
```

Vérification avant de continuer

Trois tests passent. Seul celui de la ligne de code échoue encore, car il attend la règle 6, écrite à l'étape suivante.

```
test_texte.py::test_tokeniser_ligne_de_test PASSED
test_texte.py::test_tokeniser_ligne_de_code FAILED
test_texte.py::test_tokeniser_ticket PASSED
test_texte.py::test_tokeniser_protege_commentaires_et_chaines PASSED
```

La dernière ligne annonce **1 failed**, **3 passed**.

Étape 4.4 — La règle 6

Étape 4.4. Ajouter la règle 6, les composants d'un identifiant (**slides 18 et 19**).

Le dernier test attend que `is_valid_email` donne aussi `is`, `valid` et `email`. **Cette règle ne s'écrit pas dans le motif, car elle ne découpe pas le texte, elle ajoute des tokens après ceux que le motif a trouvés.** Remplacez la ligne `return` de l'étape 4.3 par les six lignes suivantes. Les tirets sont à remplacer par le geste de la règle 6, écrit dans la colonne « son écriture » du **slide 18**.

```
tokens = []
for token in re.findall(MOTIF, texte):
    tokens.append(token)
    if "_" in token and re.fullmatch(r"\w+", token):
        tokens.extend(c for c in _____ if c)
return tokens
```

Étape 4.4 — La règle 6 (suite)

- `tokens = []` crée une liste vide qui recevra le résultat.
- La boucle `for` prend un par un les tokens trouvés par le motif, et `append` range chacun dans la liste.
- La ligne `if` demande si le token contient un tiret bas **et** s'il est un mot. **Un commentaire qui contient un tiret bas n'est pas un mot, il n'est donc pas décomposé.**
- `extend` ajoute les composants juste après le token. Le `if` final écarte les composants vides.

Enregistrez.

Étape 4.5 — Les quatre tests de la partie

Étape 4.5. Lancer les quatre tests de la partie.

```
pytest -v -k tokeniser
```

Vérification avant de continuer

```
test_texte.py::test_tokeniser_ligne_de_test PASSED
test_texte.py::test_tokeniser_ligne_de_code PASSED
test_texte.py::test_tokeniser_ticket PASSED
test_texte.py::test_tokeniser_protege_commentaires_et_chaines PASSED
===== 4 passed, 9 deselected in 0.12s =====
```

Étape 4.6 — Une ligne du fil par toute la chaîne

Étape 4.6. Faire passer une ligne du fil par toute la chaîne.

Relancez python. La fonction **chaîne**, déjà écrite en bas de **texte.py**, enchaîne vos quatre fonctions.

```
>>> from texte import chaîne
>>> chaîne("donnees/ligne_code.txt")
['def', 'is_valid_email', 'is', 'valid', 'email', '(', 'adresse', ')', ':',
 '# créé par K. Alaoui']
>>> chaîne("donnees/ticket_darija.txt")
['l', '"', 'app', 't7bes', ',', 'surface', 'en', 'm2', 'fausse']
>>> exit()
```

Le fichier contenait « **crÃ©Ã©** » et « **m²** ». Les tokens contiennent « **créé** » et **m2**.
Les quatre étapes ont travaillé l'une après l'autre.

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Lire
- 3 Partie 2 — Réparer
- 4 Partie 3 — Normaliser
- 5 Partie 4 — Tokeniser
- 6 Partie 5 — Mesurer**
- 7 Partie 6 — Publier

Lien avec le cours

Cette partie applique les **slides 21** (La loi de Zipf) et **22** (La loi de Heaps). L'étape 5.1 est l'encadré Application du **slide 21**, l'étape 5.3 produit sur votre machine les deux tableaux des **slides 21 et 22**.

Étape 5.1 — La fonction `mesurer`

Étape 5.1. Compléter la fonction `mesurer` (slide 21, encadré Application).

Remplacez la ligne raise de `mesurer` par une seule ligne. `Counter` est déjà importé en haut du fichier, il compte combien de fois chaque élément apparaît dans une liste.

```
return Counter(_____)
```

Remplacez les tirets par le nom de l'argument de la fonction, puis enregistrez.

Étape 5.2 — Tous les tests

Étape 5.2. Lancer tous les tests.

```
pytest -v
```

Vérification avant de continuer

Treize lignes se terminent par **PASSED**, et la dernière ligne est la suivante.

```
===== 13 passed in 0.08s =====
```

La chaîne est complète et chaque fonction est prouvée.

Étape 5.3 — Un vrai corpus

Étape 5.3. Mesurer un vrai corpus (slides 21 et 22).

Le script **mesure_corpus.py** fait passer par votre chaîne tous les fichiers de la bibliothèque standard de Python, déjà présente sur votre machine, puis compte les mots.

```
python mesure_corpus.py
```

Étape 5.3 — Un vrai corpus (suite)

Vérification avant de continuer

L'écran affiche deux tableaux. Voici ceux obtenus avec Python 3.12. Vos nombres peuvent différer un peu si votre version de Python est différente, l'allure doit être la même.

rang	token	fréquence	rang x fréquence
1	self	19585	19585
2	if	9705	19410
3	return	6704	20112
10	name	3269	32690
100	Modules	465	46500

Étape 5.3 — Un vrai corpus (fin)

Vérification avant de continuer

mots lus N	vocabulaire V
1000	299
10000	1433
100000	7804
349886	21847

Étape 5.4 — Vos propres chiffres

Étape 5.4. Lire vos propres chiffres.

Répondez sur votre feuille, avec **vos** nombres.

- 1 Divisez la fréquence du rang 1 par celle du rang 100. Par combien la fréquence a-t-elle été divisée ? Le produit de la dernière colonne a-t-il changé autant ? C'est la **loi de Zipf** du **slide 21**.
- 2 Entre 100 000 mots lus et la fin du corpus, combien de mots nouveaux sont apparus ? Le vocabulaire a-t-il fini de grandir ? C'est la **loi de Heaps** du **slide 22**.

Dans une autre application

Type d'application. Toute application souveraine, avant d'être construite, sur les comptes rendus d'un hôpital comme sur les contrats d'une banque.

Comment. Dans `mesure_corpus.py`, la ligne `DOSSIER = ...` désigne le corpus. En la remplaçant par le dossier des documents de l'entreprise, le même script donne le premier contrôle du corpus. Les premiers rangs doivent montrer le vocabulaire du métier. Si des débris comme **Ã** y apparaissent, l'étape Réparer a été oubliée. Si le vocabulaire grandit toujours, l'application devra savoir traiter un mot jamais vu, ce que construit la séance 2.

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Lire
- 3 Partie 2 — Réparer
- 4 Partie 3 — Normaliser
- 5 Partie 4 — Tokeniser
- 6 Partie 5 — Mesurer
- 7 Partie 6 — Publier**

Lien avec le cours

Cette partie applique les **slides 24** (Le trajet complet du texte) et **25** (Le premier maillon est construit). Les trois commandes sont celles du cadre gris du **slide 25**.

Étapes 6.1 et 6.2 — Les fichiers à enregistrer

Étape 6.1. Regarder ce qui a changé.

Vous êtes toujours dans le dossier **seance1**.

```
git status
```

Git affiche en rouge, sous Untracked files, le dossier de la séance. **Il le voit mais ne l'enregistre pas encore.**

Étape 6.2. Choisir les fichiers à enregistrer.

```
git add texte.py test_texte.py mesure_corpus.py donnees
```

Cette commande n'affiche rien. Elle place les trois fichiers et le dossier **donnees** dans la liste de ce qui sera enregistré. **Refaites git status, ils apparaissent maintenant en vert.**

Étape 6.3 — Le *commit*

Étape 6.3. Enregistrer une version, le *commit*.

```
git commit -m "Séance 1, du fichier aux tokens, 13 tests verts"
```

Le texte entre guillemets est le message qui décrit la version. Git répond par une ligne qui commence par [main suivie de votre message, puis par le nombre de fichiers enregistrés.

Étape 6.4 — L'envoi sur GitHub

Étape 6.4. Envoyer la version sur GitHub.

```
git push
```

Vérification avant de continuer

La dernière ligne affichée contient `main -> main`. Ouvrez la page de votre dépôt **iliacode** sur GitHub et rafraîchissez-la. Le dossier **seancel** y apparaît, avec votre message de commit à côté.